Homework 9

Please complete the following problems. You must submit the following for each problem: a) the answer you are asked to return and b) Python code that can be run to verify your work.

1. Enumerating k-mers Lexicographically

When cataloguing a collection of genetic strings, you should have an established system by which to organize them. The standard method is to organize strings as they would appear in a dictionary, so that "APPLE" precedes "APRON", which in turn comes before "ARMOR".

PROBLEM

Assume that an alphabet $\mathcal{A}$ has a predetermined order; that is, we write the alphabet as a permutation $\mathcal{A} = (a_1, a_2, \ldots, a_k)$, where $a_1 < a_2 < \cdots < a_k$. For instance, the English alphabet is organized as $(A, B, \ldots, Z)$.

Given two strings s and t having the same length n, we say that s precedes t in the lexicographic order (and write $s <_{Lex} t$) if the first symbol $s[j]$ that doesn't match $t[j]$ satisfies $sj < tj$ in $\mathcal{A}$.

Given: A file containing a collection of at most 10 symbols defining an ordered alphabet, and a positive integer n ($n \leq 10$).

Return: All strings of length n that can be formed from the alphabet, ordered lexicographically (use the standard order of symbols in the English alphabet).

Sample Dataset

```
A C G T
2
```

Sample Output

```
AA
AC
AG
AT
CA
CC
CG
CT
GA
GC
GG
GT
TA
TC
TG
TT
```

Hint: There are multiple ways to solve this problem. You can build a recursive algorithm with loops. Or you can import `itertools` and use `itertools.product()` to create the list in a single line.

2. <u>k-Mer Composition</u>

A length k substring of a genetic string is commonly called a k-mer. A genetic string of length n can be seen as composed of $n - k + 1$ overlapping k-mers. The k-mer composition of a genetic string encodes the number of times that each possible k-mer occurs in the string. See Figure 1. The 1-mer composition is a generalization of the GC-content of a strand of DNA, and the 2-mer, 3-mer, and 4-mer compositions of a DNA string are also commonly known as its di-nucleotide, tri-nucleotide, and tetra-nucleotide compositions.

AA	AC	AG	AT	CA	CC	CG	CT	GA	GC	GG	GT	TA	TC	TG	TT
3	5	0	8	6	2	1	3	2	2	0	4	5	3	7	8

Figure 1: The 2-mer composition of TTGATTACCTTATTTGATCATTACACATTGTA.

The biological significance of k-mer composition is manyfold. GC-content is helpful not only in helping to identify a piece of unknown DNA (see the "Computing GC Content" problem), but also because a genomic region having high GC-content compared to the rest of the genome signals that it may belong to an exon. Analyzing k-mer composition is vital to fragment assembly as well.

In the "Computing GC Content" problem, an analogy was drawn between analyzing the frequency of characters and identifying the underlying language. For larger values of k, the k-mer composition offers a more robust fingerprint of a string's identity because it offers an analysis on the scale of substrings (i.e., words) instead of that of single symbols. As a basis of comparison, in language analysis, the k-mer composition of a text can be used not only to pin down the language, but also often the author.

<u>PROBLEM</u>

For a fixed positive integer k, order all possible k-mers taken from an underlying alphabet lexicographically. Then the k-mer composition of a string s can be represented by an array A for which $A[m]$ denotes the number of times that the mth k-mer (with respect to the lexicographic order) appears in s.

Given: A file containing a DNA string s in FASTA format (having length at most 100 kb).

Return: The 4-mer composition of s.

Sample Dataset

```
>Rosalind_6431
CTTCGAAAGTTTGGGCCGAGTCTTACAGTCGGTCTTGAAGCAAAGTAACGAACTCCACGG
CCCTGACTACCGAACCAGTTGTGAGTACTCAACTGGGTGAGAGTGCAGTCCCTATTGAGT
TTCCGAGACTCACCGGGATTTTCGATCCAGCCTCAGTCCAGTCTTGTGGCCAACTCACCA
AATGACGTTGGAATATCCCTGTCTAGCTCACGCAGTACTTAGTAAGAGGTCGCTGCAGCG
GGGCAAGGAGATCGGAAAATGTGCTCTATATGCGACTAAAGCTCCTAACTTACACGTAGA
CTTGCCCGTGTTAAAAACTCGGCTCACATGCTGTCTGCGGCTGGCTGTATACAGTATCTA
CCTAATACCCTTCAGTTCGCCGCACAAAAGCTGGGAGTTACCGCGGAAATCACAG
```

Sample Output

```
4 1 4 3 0 1 1 5 1 3 1 2 2 1 2 0 1 1 3 1 2 1 3 1 1 1 1 2 2 5 1 3 0 2 2 1 1 1 1 3 1 0 0 1 5 5 1
5 0 2 0 2 1 2 1 1 1 2 0 1 0 0 1 1 3 2 1 0 3 2 3 0 0 2 0 8 0 0 1 0 2 1 3 0 0 0 1 4 3 2 1 1 3 1
2 1 3 1 2 1 2 1 1 1 2 3 2 1 1 0 1 1 3 2 1 2 6 2 1 1 1 2 3 3 3 2 3 0 3 2 1 1 0 0 1 4 3 0 1 5 0
2 0 1 2 1 3 0 1 2 2 1 1 0 3 0 0 4 5 0 3 0 2 1 1 3 0 3 2 2 1 1 0 2 1 0 2 2 1 2 0 2 2 5 2 2 1 1
2 1 2 2 2 2 1 1 3 4 0 2 1 1 0 1 2 2 1 1 1 5 2 0 3 2 1 1 2 2 3 0 3 0 1 3 1 2 3 0 2 1 2 2 1 2 3
0 1 2 3 1 1 3 1 0 1 1 3 0 2 1 2 2 0 2 1 1
```

Hint: You may use the same strategy as you chose to solve the "Enumerating k-mers Lexicographically" problem to obtain all the available 4-mers. With this in hand, there are multiple strategies to count the 4-mers in the given sequence. One efficient strategy is to create an empty dictionary that will hold each k-mer as a key and the number of times the k-mer appears as a value.

3. Constructing a De Bruijn Graph

Because you use multiple copies of the genome to generate and identify reads for the purposes of fragment assembly, the total length of all reads will be much longer than the genome itself. This begs the definition of read coverage as the average number of times that each nucleotide from the genome appears in the reads. In other words, if the total length of our reads is 30 billion bp for a 3 billion bp genome, then we have 10x read coverage.

To handle so many k-mers for the purposes of sequencing the genome, we need an efficient and simple structure. The de Bruijn graph is a directed graph used for representing overlapping strings in a collection of k-mers.

PROBLEM

Consider a set S of $(k+1)$-mers of some unknown DNA string. Let S^{rc} denote the set containing all reverse complements of the elements of S. (Recall that sets are not allowed to contain duplicate elements).

The de Bruijn graph B_k of order k corresponding to $S \cup S^{rc}$ is a digraph defined in the following way:

- Create nodes of B for any length k substring of some $(k+1)$-mer in $S \cup S^{rc}$.
- Do not form multiple nodes corresponding to the same DNA string.
- Create directed edges of B that connect node s to node t if there is some $(k+1)$-mer u in $S \cup S^{rc}$ whose prefix is s and suffix is t. (You can label the directed edge with the $(k+1)$-mer u.

Given: A file containing a collection of up to 1000 (possibly repeating) DNA strings of equal length (not exceeding 50 bp) corresponding to a set S of $(k+1)$-mers.

Return: The adjacency list corresponding to the de Bruijn graph corresponding to $S \cup S^{rc}$.

Sample Dataset

```
TGAT
CATG
TCAT
ATGC
CATC
CATC
```

Sample Output

```
(ATC, TCA)
(ATG, TGA)
(ATG, TGC)
(CAT, ATC)
(CAT, ATG)
(GAT, ATG)
(GCA, CAT)
(TCA, CAT)
(TGA, GAT)
```

Hint: You may find the `defaultdict(list)` command useful after importing it from the `collections` module. This command creates a dictionary of lists. To add elements to the dictionary, you can use the `append` function. In this problem, the dictionary keys would be the prefix k-mers, and the dictionary values would be the list of compatible suffix k-mers. Also, consider defining a function for creating a de Bruijn graph, as you may use that function in subsequent problems.